

Restricted Randomization Beyond Permuted Blocks

SCT Arizona 2026

Kevin Venner, Diane Uschner

May 2026



Introduction



Outline

1. Why block randomization is increasingly challenged
2. Mathematical perspective on restricted randomization
3. Maximum tolerated imbalance (MTI) framework
4. Practical alternatives to permuted block randomization
5. Predictability under equal MTI constraints
6. Implementation and take-home messages

Why Rethink Permuted Block Randomization?

Block Randomization Under Challenge

- ▶ Permuted block randomization has been widely used for decades because it controls treatment imbalance tightly.
- ▶ In modern trials, that strong control can come at a cost: higher allocation predictability.
- ▶ Predictability increases the risk of selection bias when treatment history can be partially inferred.
- ▶ The core question is no longer only balance, but balance versus randomness and robustness.

Dynamic Allocation and Current Trial State

- ▶ Treatment assignment is sequential and depends on the current state of the trial.
- ▶ In blocked and stratified designs, assignment probabilities are influenced by current within-block imbalance.
- ▶ Stratification adds another dynamic layer because covariate strata evolve over enrollment.
- ▶ This dynamic dependence is shared by many restricted randomization designs, not only permuted blocks.

Mathematical Framing

Restricted Randomization as a Dynamic Process

- ▶ Restricted randomization can be viewed as a stochastic process on imbalance states.
- ▶ At each step, the design updates assignment probabilities using current history.
- ▶ Design properties are therefore path-dependent, not just determined by the final sample size.
- ▶ This framing helps compare designs on predictability, imbalance, and inferential performance.

Dependence on Treatment Balance and Stratum State

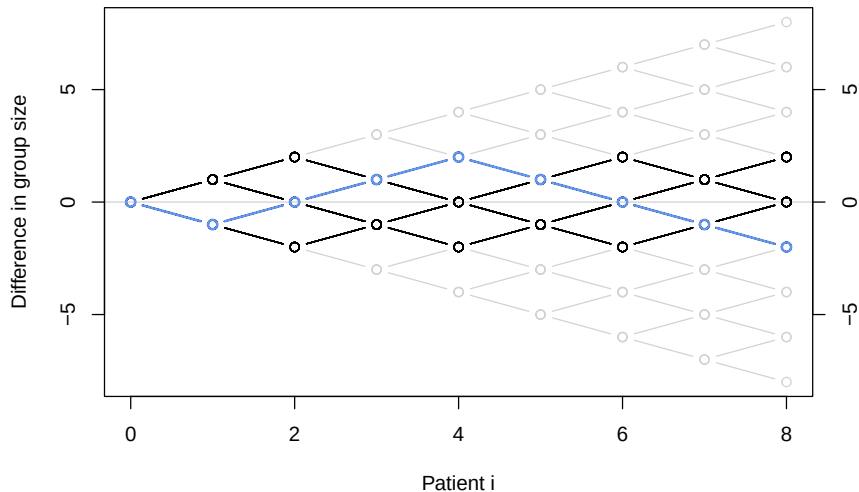
- ▶ Let $D_i = N_C(i) - N_E(i)$ denote treatment imbalance after subject i .
- ▶ In stratified settings, each stratum has its own imbalance trajectory.
- ▶ The next assignment probability is a function of the current imbalance state (and potentially stratum).
- ▶ Strong correction toward $D_i = 0$ improves balance but can make assignments easier to guess.

MTI-Based Design Strategy

Maximum Tolerated Imbalance (MTI)

- ▶ MTI designs enforce a hard boundary on allowable imbalance: $|D_i| \leq b$.
- ▶ Inside the boundary, randomization is flexible; at the boundary, correction is enforced.
- ▶ MTI provides a transparent tuning parameter for the balance-randomness trade-off.
- ▶ This gives a common framework to compare procedures at the same imbalance tolerance.

Visualization of the MTI tunnel



Releasing Enforced Block-End Perfect Balance

- ▶ Permuted blocks force deterministic structure near the end of each block.
- ▶ MTI-based alternatives relax this block-end perfect balance requirement.
- ▶ The result is lower predictability while still controlling worst-case imbalance.
- ▶ In practice, this can reduce selection bias risk without sacrificing operational simplicity.

Alternative Designs

Maximal Procedure

- ▶ The maximal procedure is an MTI-constrained restricted randomization design with equi-probable randomization sequences and the constraint of final group balance
- ▶ It maximizes randomness subject to the imbalance bound, avoiding fixed block structure.
- ▶ Under matched MTI constraints, it is less predictable than permuted block randomization.
- ▶ It is a practical option when allocation concealment and anti-predictability as well as exact final balance are priorities.
- ▶ Implementation requires knowing the target group sizes in advance.

Big Stick Design

- ▶ Big Stick Design randomizes freely until the MTI boundary is reached.
- ▶ At the boundary, assignment is forced only to move imbalance back toward zero.
 - ▶ As a result, sequences are not equi-probable, but the design is less predictable than permuted blocks.
- ▶ This yields less deterministic behavior than permuted blocks while preserving MTI control.
- ▶ Operationally, it is straightforward to implement with centralized randomization tools.

Implementation Simplicity and Practical Use

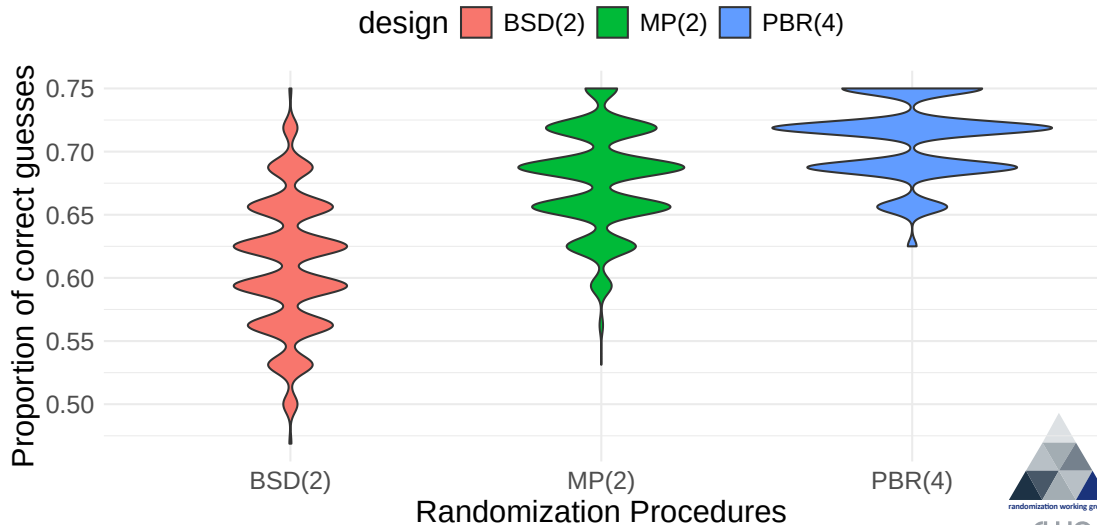
- ▶ Big Stick and maximal procedure designs are feasible for routine trial operations.
- ▶ They can be pre-specified in SAP/protocol language similarly to existing restricted methods.
- ▶ Implementation burden is typically low with modern IWRS/IRT infrastructure.
- ▶ Design choice should be justified by expected bias risk, not convenience alone.

Predictability Evidence

Equal-MTI Comparisons Across Designs

- ▶ Fair comparisons should match designs on the same MTI level.
 - ▶ For the permuted block design, the MTI is half the block size (e.g., $\text{MTI}=2$ for block size 4).
- ▶ Under equal MTI, designs can differ meaningfully in allocation predictability.
- ▶ Literature shows that predictability is not an unavoidable consequence of imbalance control.
- ▶ This supports selecting designs by properties, not by historical default.

Comparison of Predictability Metrics



Why Permuted Block Is Most Predictable Under MTI

- ▶ With fixed block structure, remaining allocations become progressively constrained.
- ▶ Near block completion, upcoming assignments can often be inferred with high probability.
- ▶ Theoretical and empirical work shows permuted blocks are most predictable among equal-MTI designs.
- ▶ Therefore, permuted blocks may be inferior when minimizing predictability is a design priority.

Conclusions

Key Messages

- ▶ Balance control and allocation unpredictability should be considered jointly.
- ▶ Under equal MTI, permuted blocks are generally more predictable than alternatives.
- ▶ Big Stick and maximal procedure are practical, implementable, and often preferable options.
- ▶ Moving beyond default blocked randomization can improve design robustness.

Discussion

- ▶ Which trial settings in our portfolio are most vulnerable to selection bias?
- ▶ What MTI level is acceptable from a clinical and operational perspective?
- ▶ Should we update internal default randomization recommendations beyond permuted blocks?